Solving Generalized Sudoku with SAT, CP-SAT and Gurobi MIP

Nguyễn Trần Hoàng Hiếu[a]

[a]VNU University of Engineering and Technology,

ARTICLE INFO

Keywords:
Sudoku
SAT solving
sequential counter
CP-SAT
mixed-integer programming

ABSTRACT

This paper presents a generalized Sudoku solver based on propositional satisfiability and compares it with OR-Tools CP-SAT and Gurobi MIP. For an $N \times N$ board, where N is a perfect square, the SAT formulation uses Boolean cell-value variables and a sequential-counter encoding of exactly-one constraints; Glucose3 is called through PySAT. The CP-SAT model uses integer cell variables and all-different constraints, while the Gurobi model uses binary assignment variables and linear exactly-one equations. We evaluate all three implementations on a small synthetic collection containing satisfiable and unsatisfiable instances, with three isolated-process repetitions and one worker or thread per method. Solutions are checked independently against clues and Sudoku rules. The experiments cover sizes up to 49×49 for all three solvers; a nearly completed 100×100 board is also evaluated with CP-SAT alone. These results demonstrate the implementations on the tested instances but are not a representative benchmark of difficult Sudoku or evidence that one approach is generally faster. SAT and Gurobi on the 100×100 case and comparisons with other exact solvers remain future work.

1. Introduction

Sudoku asks for a completion of a partially filled grid such that each row, column, and block contains each symbol exactly once. Its finite domains and interacting constraints make it a useful example for studying logical encodings and exact solving. SAT-based Sudoku encodings have been studied by Lynce and Ouaknine [1], who show how formulation choices affect inference. PySAT provides Python interfaces to several SAT solvers [3], while OR-Tools CP-SAT offers an alternative interface for integer constraint models [4].

This project extends an existing 9×9 solver to generalized square-block Sudoku. It contributes an implementation of a sequential-counter SAT encoding, separate CP-SAT and Gurobi MIP models for comparison, independently checked synthetic test instances, and a reproducible preliminary experiment. The experiment compares the three implemented formulations, rather than isolating a single algorithmic component. Because the dataset is small and largely derived from one grid pattern, the results should not be generalized to arbitrary Sudoku distributions. CPLEX CP and CPLEX MIP, also mentioned in the course requirement, are outside the present comparison.

2. Problem definition

Let $N = k^2$ for a positive integer k. The input is an $N \times N$ matrix P, whose entries lie in $\{0, 1, \dots, N\}$; zero denotes an empty cell. The task is to find $Y \in \{1, \dots, N\}^{N \times N}$ such that every row, every column, and every $k \times k$ block contains all N values, and $Y_{r,c} = P_{r,c}$ whenever $P_{r,c} \neq 0$. If no such matrix exists, the puzzle is unsatisfiable. Malformed inputs, such as non-square rows or values outside the allowed range, are rejected and are not classified as unsatisfiable puzzles. The implementation seeks one solution; it does not test whether that solution is unique.

3. Solving methods

3.1. SAT formulation

For each row $r \in \{0, \dots, N-1\}$, column $c \in \{0, \dots, N-1\}$ and value $v \in \{1, \dots, N\}$, introduce a Boolean variable $x_{r,c,v}$, true if and only if cell (r, c) contains v. Its positive integer identifier is $N^2 r + Nc + v$. Exactly-one (EO) constraints are imposed on the N values of each cell, the N cells of each row for each value, the N cells of each column for each value, and the N cells of each block for each value. For a clue $P_{r,c} = v$, the unit clause $(x_{r,c,v})$ fixes the corresponding value. Each EO constraint is the conjunction of an at-least-one clause and an at-most-one (AMO) encoding. The AMO construction uses sequential auxiliary variables, following the sequential-counter idea of Sinz

[2]. For inputs $z_1, \ldots, z_N$ with $N \geq 2$, it introduces $s_1, \ldots, s_{N-1}$ and uses

$$(\neg z_1 \vee s_1) \wedge (\neg z_N \vee \neg s_{N-1}) \tag{1}$$

$$\wedge \bigwedge_{i=2}^{N-1} [(\neg z_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg z_i \vee \neg s_{i-1})]. \tag{2}$$

Intuitively, the auxiliary chain records whether an earlier input is true; a later input cannot also be true. The resulting formula has a satisfying assignment exactly when the Sudoku instance has a completion: the EO constraints enforce the Sudoku rules and clues, while any valid grid can be extended to the auxiliary variables. The solution is decoded from the true primary variables.

For $N \geq 2$ there are N^3 primary variables and $4N^2$ EO groups. Each group uses $N - 1$ auxiliary variables and $3N - 3$ clauses including the at-least-one clause. Thus, with G clues, the generated formula has $N^3 + 4N^2(N - 1)$ variables and $4N^2(3N - 3) + G$ clauses. The $N = 1$ implementation is a special case without auxiliary variables. Although the generator streams clauses to Glucose3 instead of holding a separate complete CNF list in Python, the solver still stores internal data; streaming does not eliminate the substantial memory requirements of large instances.

3.2. CP-SAT comparison model

The second model declares integer variables $Y_{r,c} \in \{1, \ldots, N\}$ directly. It imposes $3N$ all-different constraints, one on each row, column and block, and fixes every clue through an equality. For this finite domain and group size, all-different implies that each group contains each value exactly once. OR-Tools CP-SAT combines SAT-based reasoning with constraint and integer programming techniques [4]. The code requests one worker for the comparisons. A feasible solution maps to SAT, and the native INFEASIBLE status maps to UNSAT. An UNKNOWN status, for example after a time limit, is not treated as proof of unsatisfiability. This is a feasibility model without an optimization objective; a solver's OPTIMAL status here should not be interpreted as a Sudoku objective value.

3.3. Gurobi MIP comparison model

The third model uses binary assignment variables $z_{r,c,v}$, equal to one when cell (r, c) contains value v. It imposes the same four families of exactly-one constraints as the SAT model, but expresses each as a linear equality $\sum z = 1$, and adds one equality $z_{r,c,v} = 1$ for each clue. Thus the input model has N^3 binary variables and $4N^2 + G$ linear equations for G clues. Gurobi [5] solves this feasibility MIP with one thread and stops after its first solution. A positive solution count maps to SAT, including when a time limit is reached with an incumbent; the native INFEASIBLE status maps to UNSAT. Other statuses without a solution remain UNKNOWN. Model construction time includes starting the licensed Gurobi environment. No objective value or uniqueness claim is inferred.

4. Experimental study

4.1. Instances and correctness checks

All evaluated cases are from the project's synthetic dataset. Satisfiable boards are created by a deterministic cyclic completed grid, optionally shifted in value, followed by deletion of cells using a fixed random seed. Their original completed grids are stored as witnesses, but deletion does not establish uniqueness. Four 4×4 unsatisfiable cases include three with immediate duplicate clues and one without an immediate duplicate. For the latter, the project's independent exhaustive check enumerates all 288 valid 4×4 completed boards and finds none compatible with the clues. Invalid-input fixtures are tested separately and excluded from timing comparisons.

The comparison includes the satisfiable cases of sizes 4, 9, 16, 25, 36 and 49, plus the four unsatisfiable 4×4 instances. Every returned SAT solution is checked independently for row, column, block and clue consistency; UNSAT outcomes are checked against the expected fixture label. The latter is supported by explicit contradictions or exhaustive enumeration, rather than by cross-solver agreement alone. Each selected case is run three times per method in fresh processes. Method order is reversed on alternating repetitions. The 100×100 instance has one blank and is run only with CP-SAT; SAT and Gurobi are not run on that input due to their large proposed input models. These are unmeasured cases, not timeouts or UNSAT results.

4.2. Environment and measurement protocol

The experiments use Python 3.12.14 on Windows 11 with an AMD Ryzen 7 8845H processor, python-sat 1.9.dev16, OR-Tools 9.15.6755 and Gurobi 13.0.3. Exact environment metadata and individual observations are supplied with the

Table 1
Median model plus solve time in seconds over three repetitions. SAT denotes Glucose3; Gurobi uses a binary MIP; NR denotes not run.

Instance	N	Blanks	Status	SAT (s)	CP-SAT (s)	Gurobi (s)
04_blank_8	4	8	SAT	0.0006	0.0052	0.0097
04_blank_12	4	12	SAT	0.0005	0.0050	0.0110
04_blank_16	4	16	SAT	0.0004	0.0107	0.0096
09_blank_40	9	40	SAT	0.0050	0.0071	0.0159
09_blank_60	9	60	SAT	0.0049	0.0203	0.0206
16_blank_128	16	128	SAT	0.0301	0.0151	0.0436
16_blank_192	16	192	SAT	0.0314	0.1010	0.0594
25_blank_200	25	200	SAT	0.1285	0.0163	0.1245
25_blank_400	25	400	SAT	0.1877	0.2440	10.3765
36_blank_360	36	360	SAT	0.4185	0.1382	0.4317
49_blank_490	49	490	SAT	1.0604	0.9954	1.5177
100_blank_1	100	1	SAT	NR	0.1836	NR
04_unsat_row	4	14	UNSAT	0.0006	0.0134	0.0133
04_unsat_column	4	14	UNSAT	0.0006	0.0037	0.0173
04_unsat_box	4	14	UNSAT	0.0005	0.0225	0.0101
04_unsat_no_duplicate	4	10	UNSAT	0.0004	0.0039	0.0101

report. Each process has a 30-second external wall-clock limit covering process startup and the entire run; this is distinct from solver-only time limits. A timeout, error or unknown result is recorded explicitly rather than assigned a zero time or an UNSAT status. The recorded model and solve times are elapsed wall-clock measurements, and the table displays their sum; program startup, input file loading, result decoding and verification are outside the displayed total. For SAT, model time includes generation and insertion of clauses into Glucose3. For CP-SAT, model time covers construction of the Python model; compilation and presolve inside the solver call are included in solve time. For Gurobi, model time also includes environment startup and license validation. Consequently the internal phases are not directly equivalent, and the total is the more appropriate comparison for this experiment. System load was not controlled, and short runs may be affected by scheduling noise.

4.3. Results

Table 1 reports the median over three runs for each instance and method. The raw CSV contains 138 runs across 16 instances; every run returned the expected status, and every reported SAT completion passed the independent checker. On the 25×25 puzzle with 400 empty cells, median times were 0.1877 s for SAT, 0.2440 s for CP-SAT, and 10.3765 s for Gurobi. On the 49×49 example, they were 1.0604 s, 0.9954 s and 1.5177 s respectively. CP-SAT's 49×49 runs varied substantially, from 0.0701 s to 1.2508 s; these observations are too few to support a stable relative-speed claim. The 100×100 CP-SAT median was 0.1836 s, while SAT and Gurobi were not run. The dataset contains only one or two examples at most of several larger sizes. The 100×100 result establishes only that this particular almost-complete puzzle was solved, not that all three solvers can handle difficult 100×100 puzzles. The data show the costs of the three modeled approaches under the stated measurement boundaries, not a general ranking across all Sudoku inputs.

The structure of the models helps interpret the scaling results. At $N = 100$, the SAT construction has one million primary and 3.96 million auxiliary variables; the structural constraints alone generate 11.88 million clauses, before 9,999 clue clauses. The corresponding Gurobi input model would have one million binary variables and 40,000 structural linear equations, plus 9,999 clue equations. In contrast, the CP-SAT model presented through the API has 10,000 integer variables and 300 all-different constraints, plus the clue equalities. These are input-model counts; solvers can create additional internal representations, so they do not describe total memory use. The results do not measure peak memory, and no conclusion about memory consumption can be drawn from timings alone.

4.4. Limitations and threats to validity

The satisfiable instances share a synthetic base construction and are not independent samples of published hard puzzles. The larger cases are sparse in number and in some cases almost complete, so board size alone should not be equated with difficulty. Repeated measurements of the same puzzle estimate timing variation, not diversity across

instances. No experimental comparison of AMO encodings, independent UNSAT proof checking, uniqueness analysis, peak memory measurement, or broad time-limit study is included. Only one formulation and worker setting is evaluated for each baseline; model design and solver versions can materially affect outcomes. Additional CPLEX CP and CPLEX MIP baselines are named in the assignment but are not implemented in this report. The Gurobi license requires online validation in the present environment, and its startup is included in measured model time.

5. Conclusion

The project implements generalized Sudoku using a sequential-counter SAT encoding with PySAT/Glucose3, a CP-SAT integer model and a Gurobi binary MIP. On the small synthetic collection, the tested outcomes agree with the known labels and checked solutions; the measured comparisons extend to 49×49, with a separate nearly completed 100×100 CP-SAT example. These are preliminary evidence of functional correctness and behavior on the supplied instances, not a claim of superiority. Further work should include established and more diverse benchmark sets, harder large puzzles, peak-memory measurements, comparisons among AMO encodings, and CPLEX CP and MIP baselines.

Data availability and reproducibility

The code, synthetic instances, benchmark script, environment metadata, and raw results are available in the project repository at `https://github.com/YoungMilos/Sudoku_pySAT_solver`, under `src/`, `data/test_puzzles/`, and `reports/sudoku_cp_sat_report/` once the current report files are committed and pushed. From the project root, run `pythonreports/sudoku_cp_sat_report/benchmark.py` after installing the dependencies in `requirements.txt`. This regenerates the CSV and may produce different timings; run `make_table.py` in the report directory to update the printed table before recompiling.

Author and course information

Nguyễn Trần Hoàng Hiếu; student ID 24021477; class 2627I_INT3011E_3; supervisor Tô Văn Khánh.

Declaration of AI-assisted preparation

An AI assistant was used to help draft this report and prepare the experiment. The author must review the manuscript, verify the results and references, and take responsibility for the submitted version.

References

[1] I. Lynce and J. Ouaknine, Sudoku as a SAT Problem, 9th International Symposium on Artificial Intelligence and Mathematics, 2006. https://people.mpi-sws.org/~joel/publications/sudoku05abs.html.

[2] C. Sinz, Towards an Optimal CNF Encoding of Boolean Cardinality Constraints, Principles and Practice of Constraint Programming – CP 2005, pp. 827–831, 2005. doi:10.1007/11564751_73.

[3] A. Ignatiev, A. Morgado, and J. Marques-Silva, PySAT: A Python Toolkit for Prototyping with SAT Oracles, SAT 2018, pp. 428–437, 2018. doi:10.1007/978-3-319-94144-8_26.

[4] L. Perron, F. Didier, and S. Gay, The CP-SAT-LP Solver, 29th International Conference on Principles and Practice of Constraint Programming, 2023. doi:10.4230/LIPIcs.CP.2023.3.

[5] Gurobi Optimization, LLC, Gurobi Optimizer Reference Manual, version 13.0, 2026. https://docs.gurobi.com/projects/optimizer/en/current/.